

INSTITUTO DE MATEMÁTICA E ESTATÍSTICA - USP

BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Felipe Kelemen Bernardino nUSP: 12623884

Gustavo Vaz Pinto nUSP: 13673194

Yesman Choque Mamani nUSP: 13643040

Rising of A Ball

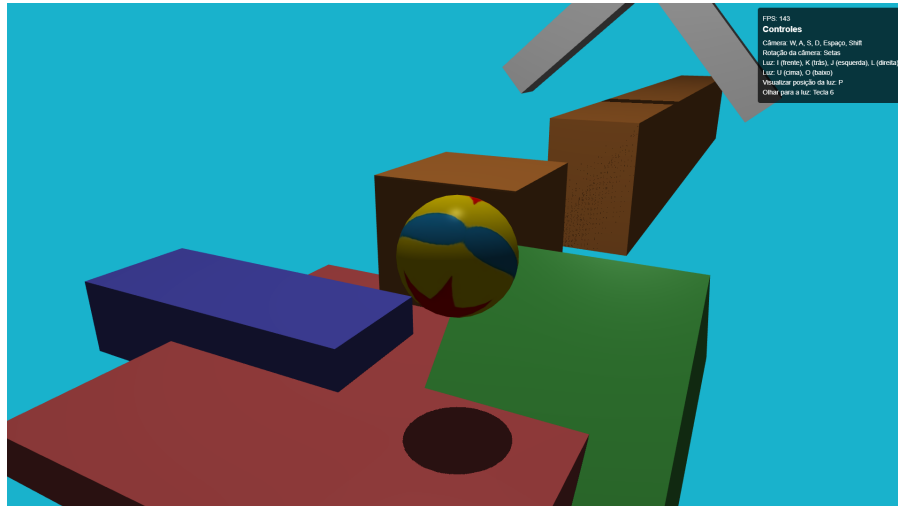
O projeto consiste em um jogo de plataforma 3D, onde o jogador controla uma bola em terceira pessoa com o objetivo de levá-la ao ponto mais alto de um circuito de obstáculos. O jogo foi desenvolvido em JavaScript com WebGL, utilizando conceitos como modelo de iluminação de Phong, movimentação física com atrito e gravidade, e simulação de quique e rotação da bola.

São Paulo

2025

1 Introdução

este é um jogo de plataforma 3D em que o jogador controla uma bola com uma perspectiva em terceira pessoa, ou seja, a bola está centrada no campo de visão. O objetivo é que o jogador percorra um circuito de obstáculos levando a bola ao ponto mais alto, através da movimentação da bola e de seus pulos. O programa foi feito em JavaScript e usa WebGL para renderizar elementos tridimensionais modelados. Para trazer mais realismo, usamos o modelo de iluminação de Phong, sombreado e movimentação da bola caracterizada pela fricção com superfícies e habilidade de quicar ao colidir com outros objetos.



2 Como Executar e Jogar

Para executar o código, basta abrir o arquivo `index.html` no navegador de sua escolha e compatível com HTML 5.

2.1 Manual

Uma vez que o jogo estiver sendo executado, o jogador pode realizar as seguintes ações:

- **Clicar na tela:** habilita a mudança de direção da câmera;
- **Mover o Mouse:** uma vez que a mudança de direção foi habilitada, o jogador pode mover a direção da câmera através do movimento do mouse;
- **Teclar Esc:** Desabilita a mudança de direção da câmera;
- **Teclar W:** Move a bola na direção em que a câmera está olhando, move para frente;
- **Teclar S:** Move a bola na direção contrária em que a câmera está olhando, move para trás;
- **Teclar A:** Move a bola na direção à esquerda em que a câmera está olhando, move para a esquerda;

- **Teclar D:** Move a bola na direção à direita em que a câmera está olhando, move para a direita;
- **Teclar Espaço:** Faz a bola pular.

3 Desenvolvimento do Sistema

3.1 Modelos e Algoritmos

3.1.1 Object3D, Mesh e Material

Cada objeto no jogo é representado por um Object3D. Cada um deles recebe um objeto de Mesh, responsável por prover ao renderizador os vértices e parâmetros como normais e posições no mapa UV. Além disso, cada objeto recebe um objeto de Material, que define as propriedades do material do objeto como cor, fator de difusão, especular e ambiente e entre outros.

3.1.2 Bola

Ela é uma extensão do Object3D que representa o jogador e suas ações. A movimentação dela é realizada de acordo com a visão da câmera, então foi feito o uso das coordenadas da câmera para transformar o vetor de velocidade da bola no eixo do mudo para o eixo de coordenadas da câmera.

Um algoritmo usado na bola para que o efeito de rolamento fosse visível foi o de calcular e aplicar a rotação de uma bola com base em sua velocidade linear no plano XY. Primeiro, ele determina a direção e intensidade do movimento para definir o eixo e a velocidade angular de rotação, simulando o efeito de rolamento realista. Se a bola estiver quase parada, aplica um fator de atrito para desacelerar a rotação residual. Quando a velocidade angular fica muito baixa, zera a rotação para evitar movimentos infinitamente pequenos. Por fim, atualiza a orientação do modelo 3D da bola usando a velocidade angular calculada, garantindo que a rotação visual corresponda ao movimento físico.

A bola aplica física básica, incluindo gravidade e atrito, para simular movimentos realistas. Ela pode ser controlada pelo jogador para se mover em todas as direções e saltar quando estiver no chão. O atrito reduz gradualmente a velocidade linear e angular, simulando desaceleração natural. Se a bola cair além de um limite definido (por exemplo, fora do cenário), ela é automaticamente reposicionada em sua posição inicial, com todas as velocidades e rotações zeradas.

3.1.3 Câmera

O objeto da câmera provê para a engine do jogo parâmetro da visão onde o mundo será projetado. Essa classe recebe parâmetros como posição, direção da visão, etc. e fornece para a engine a matriz de projeção e visualização, que serão usadas pelo vertex shader para projetar cada vértice na tela.

O algoritmo da câmera também implementa um sistema de acompanhamento suave e orientável em torno de um objeto de interesse, ou seja, a bola. Ele calcula a posição do observador (olho da câmera) em relação ao centro do objeto, utilizando coordenadas esféricas controladas pelos ângulos θ e ϕ . Esses ângulos permitem que a câmera orbite ao redor do objeto, limitando o movimento vertical para evitar ângulos extremos que poderiam comprometer a visualização.

Para garantir uma transição suave entre posições, o algoritmo interpola gradualmente a posição e o alvo da câmera usando uma função de interpolação linear, evitando movimentos bruscos. Além disso, a classe calcula e atualiza as matrizes de visualização e projeção, essenciais para transformar as coordenadas do mundo 3D para o espaço da tela, permitindo que o vertex shader projete corretamente cada vértice. O sistema também ajusta automaticamente o aspecto da projeção conforme a resolução da tela, mantendo a proporção da cena renderizada.

3.1.4 Cubo e Plataforma Móvel

O cubo é utilizado como base geométrica para criar plataformas no cenário 3D, sendo instanciado pela classe `Cube`, que define seus vértices, cores, normais e coordenadas de textura. Para gerar uma plataforma móvel, um objeto é criado com um cubo como malha (mesh) e propriedades como posição, escala e material. O movimento lateral da plataforma é controlado pela função `handleMovement(dt)`, que ajusta a posição da plataforma ao longo do eixo X de acordo com a direção e velocidade definidas, invertendo o sentido quando atinge os limites estabelecidos. Isso permite criar plataformas que se deslocam de um lado para o outro de forma contínua.

3.1.5 Moinho

Dois cubos são utilizados para representar as pás de um moinho de vento. Cada pá é criada como um objeto 3D (`Object3D`) com formato de cubo alongado (usando `scale` para deixá-lo fino e comprido) e posicionada no topo da torre do moinho. As pás são orientadas em ângulos diferentes para simular o formato clássico de um moinho. A função `handleMovement(dt)` é responsável por animar a rotação dessas pás, incrementando continuamente o valor de rotação em torno do eixo Y de cada cubo conforme o tempo passa (dt), criando assim o efeito visual de um moinho girando.



Figura 1: Moinho com cubos esticados cruzados e cubo móvel em roxo

3.1.6 Luz e Sombra

Para simular a interação da luz com os objetos da cena, utilizamos o modelo de iluminação de Phong. Esse modelo calcula a intensidade da luz que atinge cada ponto da superfície com base na posição da fonte de luz, na normal da superfície e na direção da câmera, permitindo obter componentes difusa, ambiente e especular, resultando em um sombreamento mais realista.

Para a geração de sombras, aplicamos a técnica de shadow mapping. Inicialmente, renderizamos a cena a partir da perspectiva da fonte de luz, armazenando, em um shadow map, a profundidade do objeto mais próximo à luz para cada ponto da projeção. Essa renderização é feita com uma projeção ortográfica, simulando raios de luz paralelos, como os da luz solar. No momento da renderização final da cena, cada fragmento verifica sua posição em relação ao shadow map: se estiver mais distante do que a profundidade registrada, ele é considerado em sombra; caso contrário, está iluminado.

3.2 Alterações da Proposta Inicial

Inicialmente a ideia era que, por meio de um longo circuito que fosse cada vez mais para cima, a bola alcançasse os céus dessa torre e, portanto, o cenário ao redor do jogador seriam árvores e nuvens passando. Entretanto, devido ao tempo e ao fator destoante que esse cenário causaria na trilha, não chegou a ser tão verticalizada como pensada inicialmente.

Um dos objetos especiais que havíamos pensado seria a gangorra, que possivelmente exigiria do jogador que ele fosse da ponta mais baixa para a mais alta de maneira rápida, já que ficar parado na ponta alta o faria descer, tal qual o brinquedo. Mas por complicação da física entre objeto e bola e a falta de tempo, não conseguimos implementá-la.

Por outro lado, organizando o código com uma estrutura mais clara, conseguimos inserir o realismo de sombras para a bola e objetos em relação a um ponto de luz.

4 Descrição das Atividades

4.1 Felipe

Desenvolveu o sistema de renderização básico do jogo. Lidou com a organização do jogo em Objects3D, Mesh e Materials. Implementou a renderização de texturas, iluminação e sombras.

4.2 Gustavo

Desenvolveu a lógica por trás da câmera que orbita e segue a bola. Também lidou com a movimentação da bola, em relação ao movimento do roll dela e da translação do mapa por interação de teclas do usuário. Por conta disso, foi responsável por integrar a bola e a câmera.

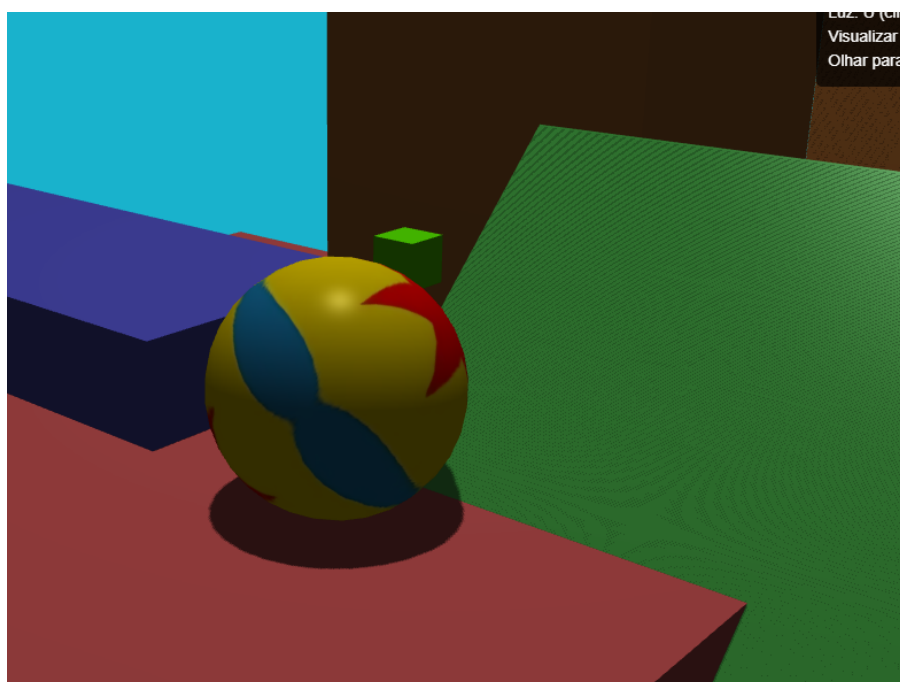
4.3 Yesman

Desenvolvimento dos objetos principais do projeto: plataformas estáticas e móveis, blocos e suas transformações, ventilador. Implementação da colisão da bola (player) com os objetos do mundo, por meio de algoritmos de colisão para AABB e OOB.

5 Bugs e Problemas Conhecidos

5.1 Shadow Acne

Na imagem a seguir, observa-se uma distorção na plataforma verde ao lado da bola, causada por um problema conhecido como shadow acne. Esse efeito ocorre quando pequenas imperfeições ou variações na geometria da superfície resultam em sombras indesejadas, que se tornam visíveis sob determinadas condições de iluminação.



5.2 Teleporte inesperado

Quando o usuário minimiza a aba do navegador durante a execução do jogo, o loop de renderização é pausado, pois o navegador suspende as atualizações visuais para páginas que não estão visíveis. Com isso, nenhuma nova requisição de renderização é processada até que a aba volte a estar ativa. Ao retornar para o jogo, o tempo acumulado entre o último frame e o atual pode ser significativamente maior que o normal, o que pode causar comportamentos inesperados, como saltos bruscos na animação, física incorreta ou lógica do jogo desajustada.